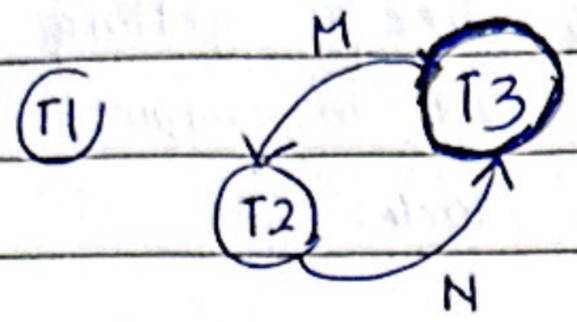
Date: / /

Question 1 :

i)

X.	Y.	M.	N.
1) R(x)	1) R(Y)	1) $R(M)^{T2} \rightarrow R(M)^{T3}$ ✗	1) $R(N)^{T2} \rightarrow R(N)^{T3}$ ✗
		2) $R(M)^{T2} \rightarrow W(M)^{T2}$ ✗	2) $R(N)^{T2} \rightarrow W(N)^{T3}$ ✓
		3) $R(M)^{T3} \rightarrow W(M)^{T2}$ ✓	3) $W(N)^{T3} \rightarrow R(N)^{T3}$ ✗



ii) cycle = yes

conflict = no

Therefore, precedence graph has a cycle, so is not serializable.

iii)

time	T1	T2	T3
t_1	begin transaction		
t_2	read_lock item(x)		
t_3	read item(x)	begin transaction	
t_4	read_lock item (Y)	read_lock item(M)	begin transaction
t_5	read item(Y)	read item (M)	read_lock item (N)
t_6	commit	read_lock item (N)	read item (N)
t_7		read item (N)	read_lock item(M)
t_8		M = (N+M) * 10	WAIT
t_9		write_lock item (M)	WAIT
t_{10}		write_item(M)	WAIT
t_{11}		commit	read item (M)
t_{12}			N = N+150
t_{13}			write_lock item (N)
t_{14}			write item (N)
t_{15}			commit

Date: / /

iv) Problem that acquire is blocking where T3 must wait until T2 release lock M and N. Next is reduced concurrency because transaction cannot interleave freely.

Potential problem is deadlock if T2 lock M and T3 lock N first, so both may wait forever. Next, is starvation, a transaction may wait too long if others keep getting locks. Lastly, livelock where no transaction are able to complete their task, and they cannot acquire any new locks.

Date: / /

Question 2.

i) P.

1) R(P)^{T4} → W(P)^{T4} (X)

Q.

1) R(Q)^{T4} → R(Q)^{T5} (X)

2) R(Q)^{T5} → W(Q)^{T5} (X)

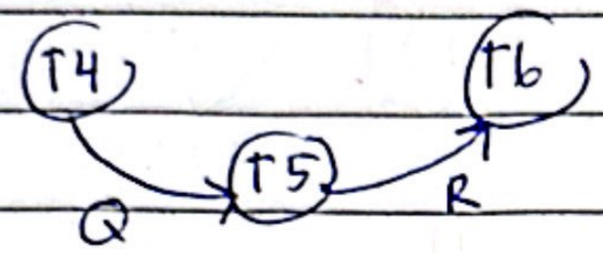
3) R(Q)^{T4} → W(Q)^{T5} (✓)

R.

1) R(R)^{T5} → R(R)^{T6} (X)

2) R(R)^{T5} → W(R)^{T6} (✓)

3) R(R)^{T6} → W(R)^{T6} (X)



ii) cycle = no

conflict = yes

∴ precedence graph has no cycle, so it has conflict ~~conflict~~ seriazable.

ii) time	T4	T5	T6
t_1	begin transaction		
t_2	read_lock item(P)		
t_3	read item(P)	begin transaction	
t_4	read_lock item(Q)	read_lock item(Q)	
t_5	read item(Q)	read item(Q)	begin transaction
t_6	P = (P * 5) + Q	read_lock item(R)	read_lock item(R)
t_7	write_lock item(P)	read item(R)	read_item(R)
t_8	write item(P)	Q = Q + R	R = R - 100
t_9	commit	write_lock item(Q)	write_lock item(R)
t_{10}		write item(Q)	write item(R)
t_{11}		commit	commit.